



Guidelines de Développement Ambulon - Version Illustrée

Documentation complète avec diagrammes PlantUML de toutes les pratiques et patterns du projet Ambulon.

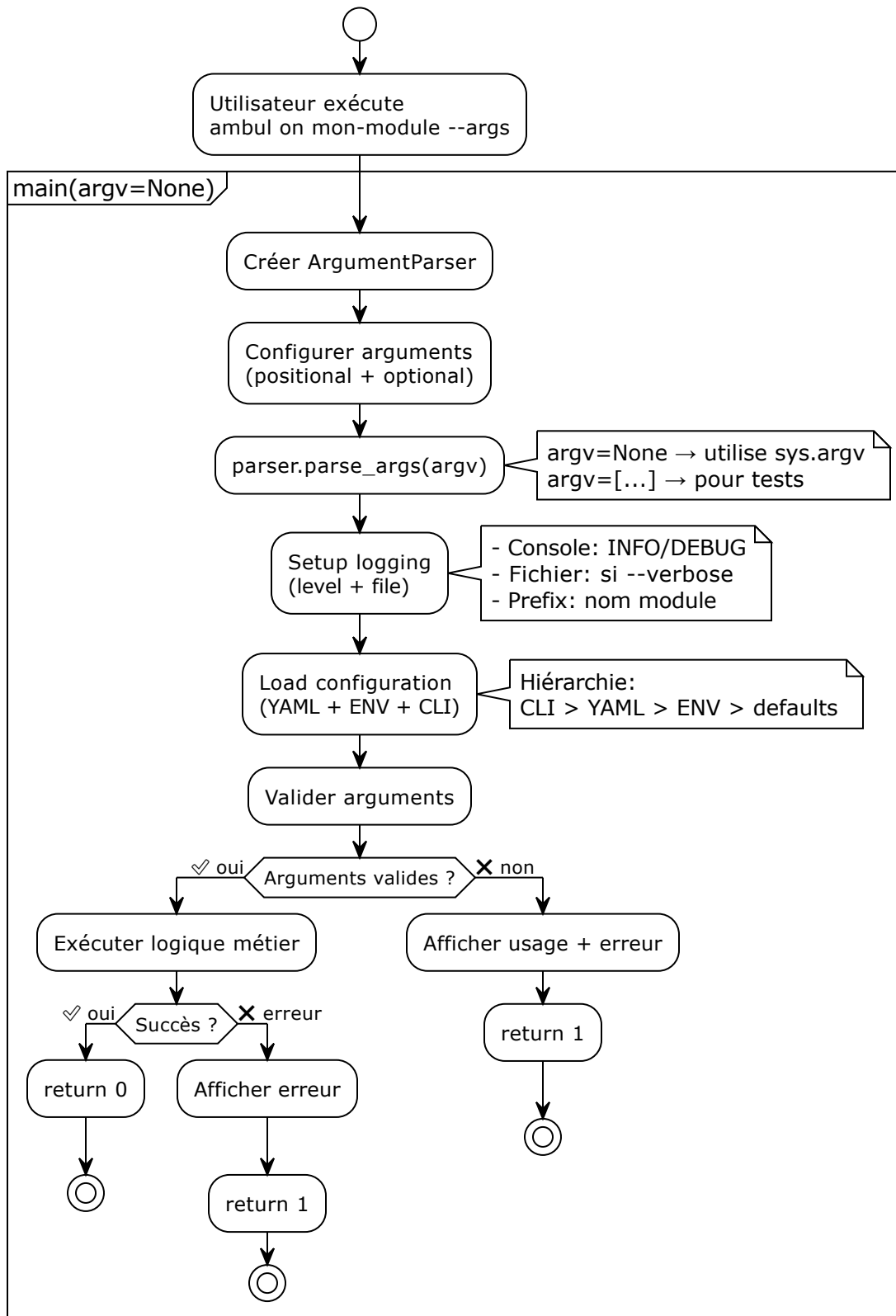
Table des Matières

1. [Architecture CLI - Interdiction Typer](#)
 2. [Pattern argparse Obligatoire](#)
 3. [Workflow Git et Branches](#)
 4. [Processus de Release](#)
 5. [Configuration Hiérarchique](#)
 6. [Pattern de Modules de Conversion](#)
 7. [Gestion des Secrets](#)
 8. [Structure du Projet](#)
 9. [Workflow Complet de Développement](#)
-

1. Architecture CLI - Interdiction Typer

Pourquoi Typer est Banni

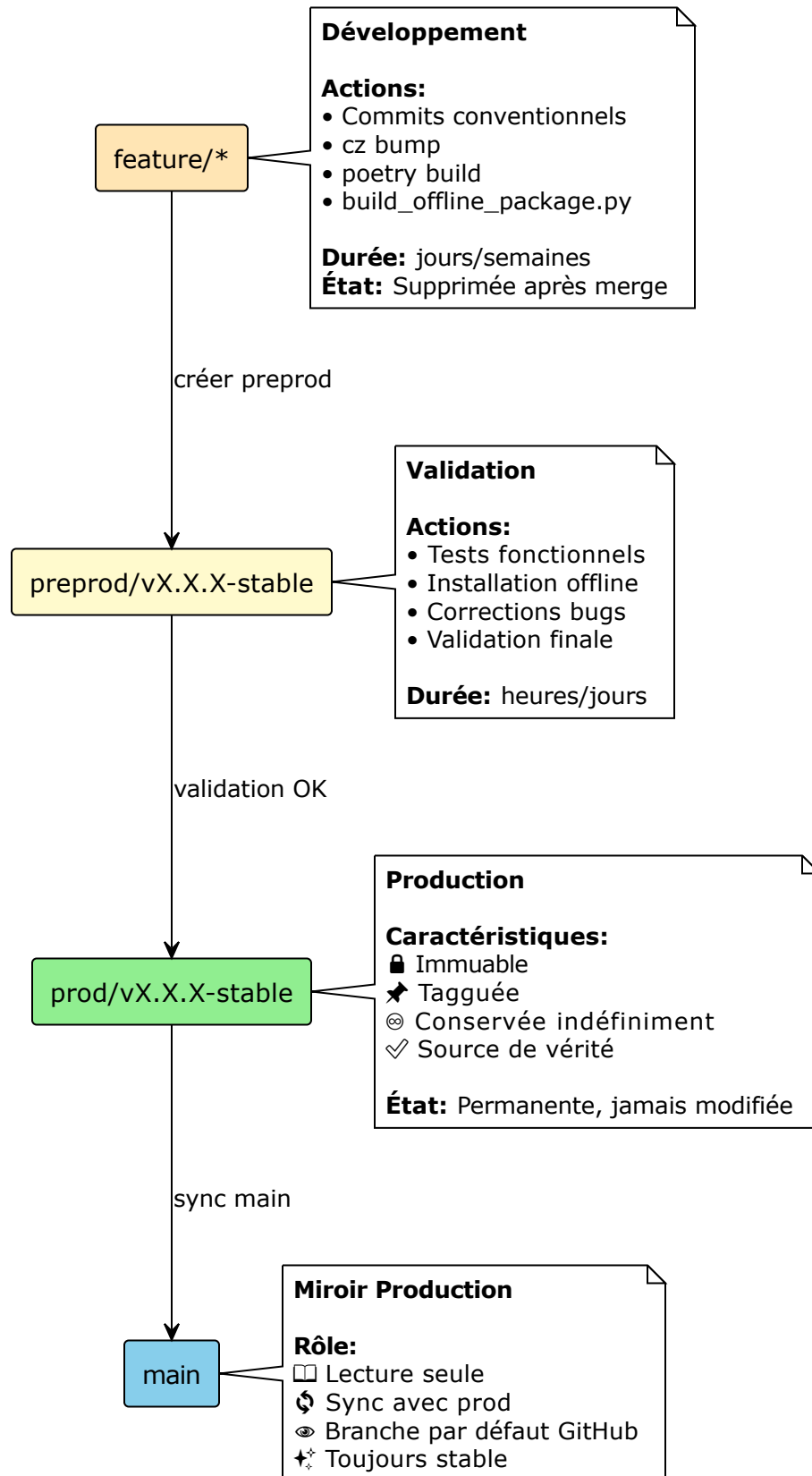
Flux d'Exécution Standard d'une Commande CLI



2. Pattern argparse Obligatoire

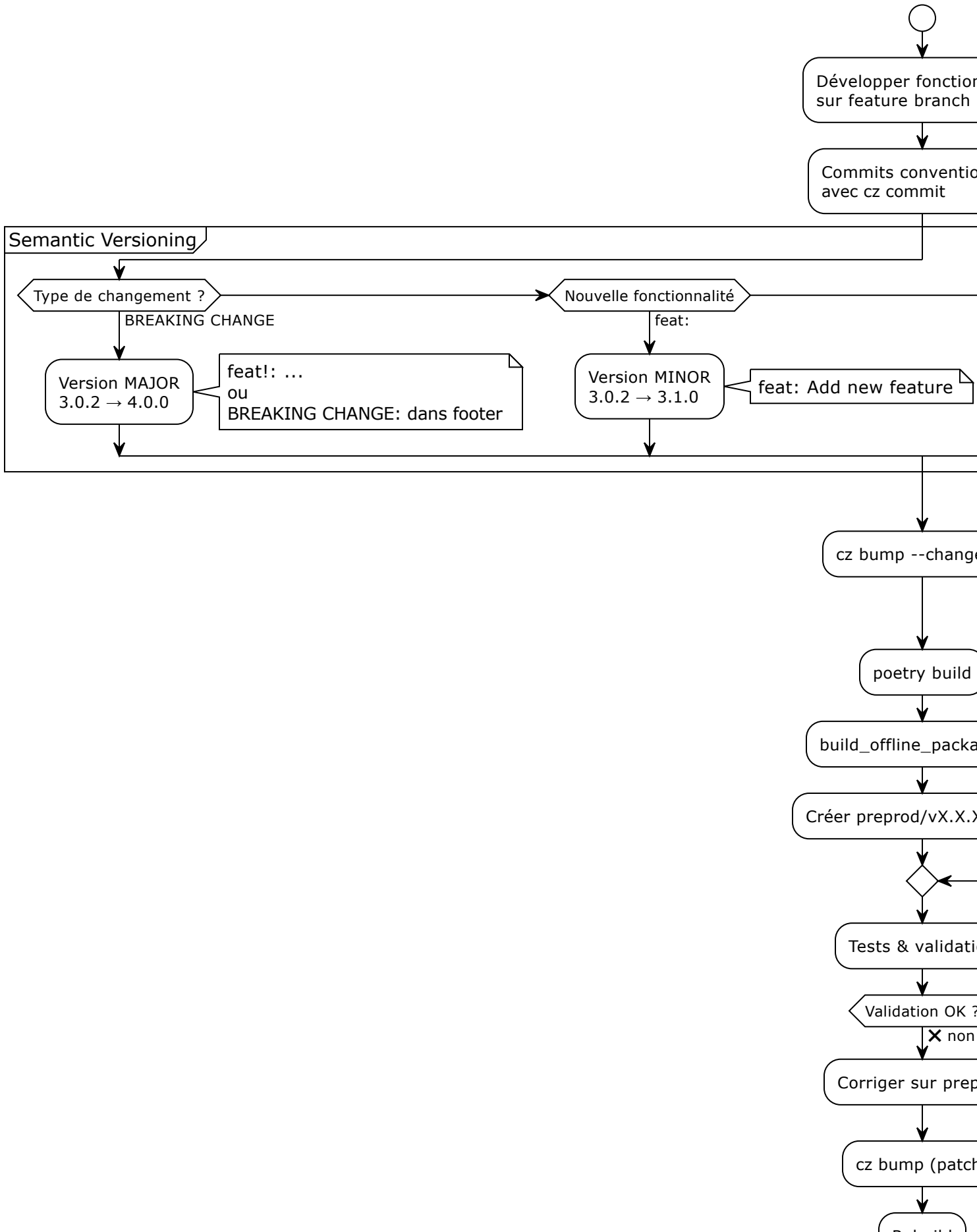
Structure d'un Module CLI

Architecture des Branches Git - Ambulon



Flux d'Exécution CLI

Processus de Release



3. Workflow Git et Branches

Architecture des Branches

Impact des Types de C

feat
Type = "feat:" Impact = MINOR Exemple = "3.0.2 → 3.1.0"

Ces commits déclenchent un bump de version via cz bump

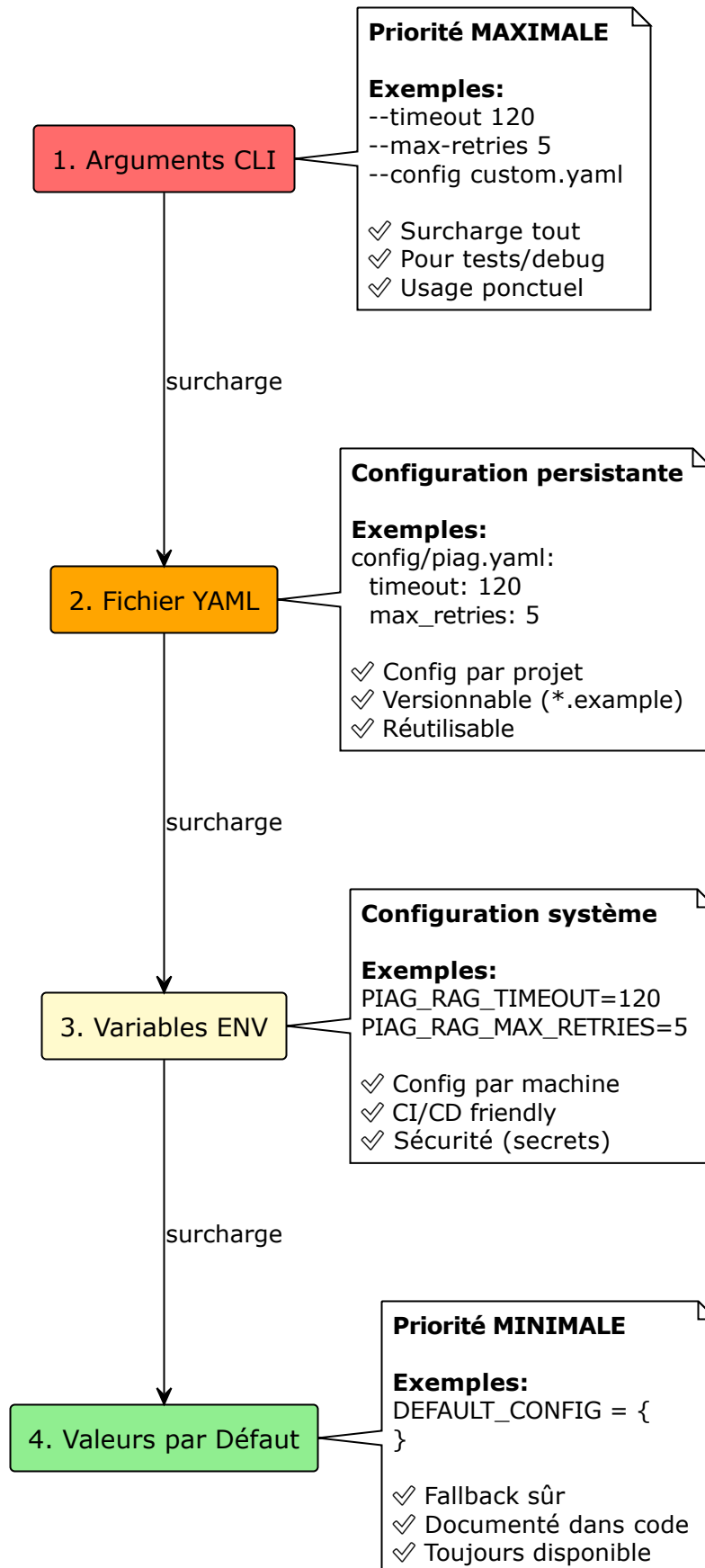
docsStyleRefactor
Types = "docs: / style: / refactor:" Impact = "Aucun" Note = "Changelog uniquement"

Voir [doc/git-workflow.md](#) pour les diagrammes détaillés du workflow Git.

4. Processus de Release

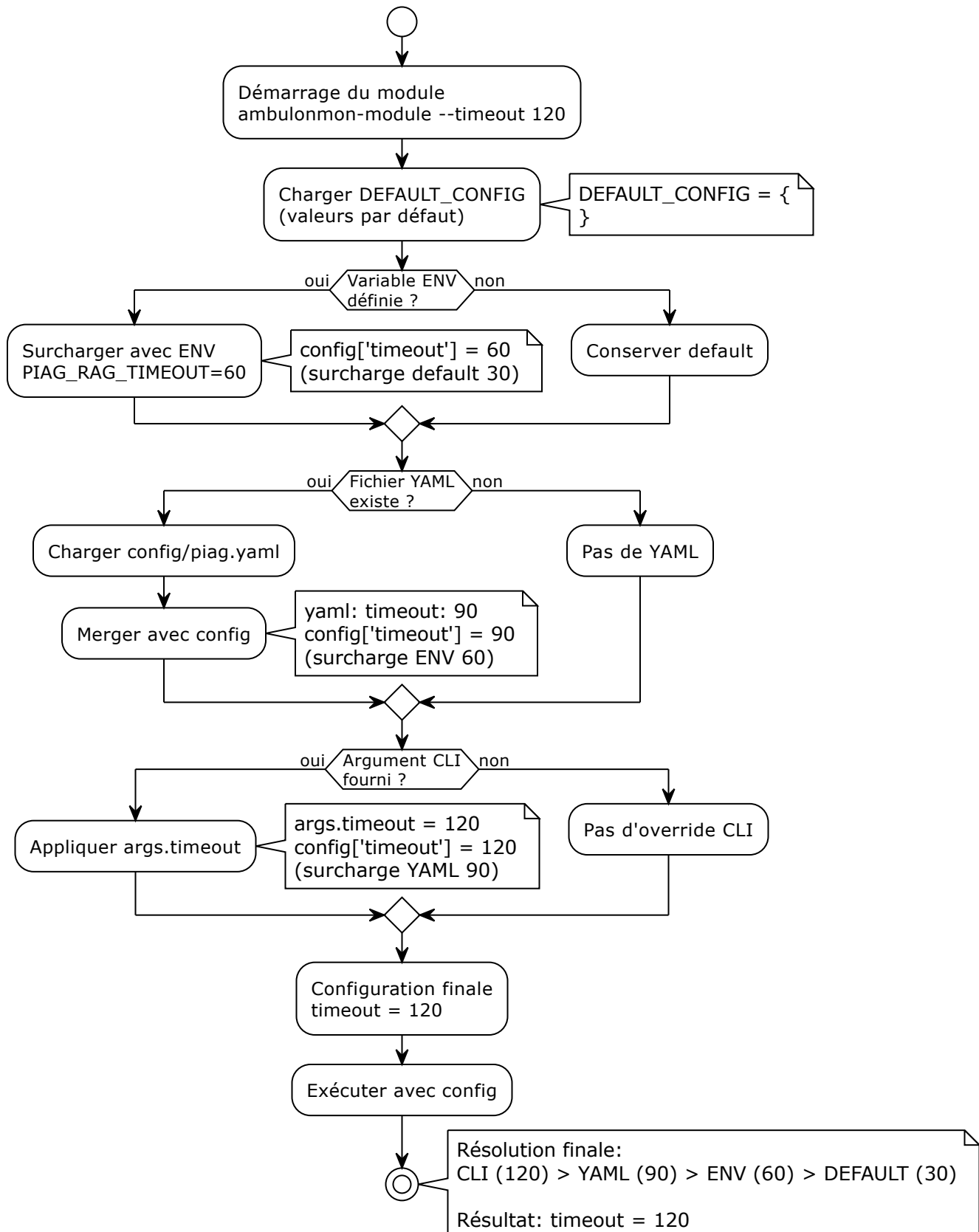
Workflow SemVer et Commitizen

Hiérarchie de Configuration - Priority Order



Types de Commits et Impact

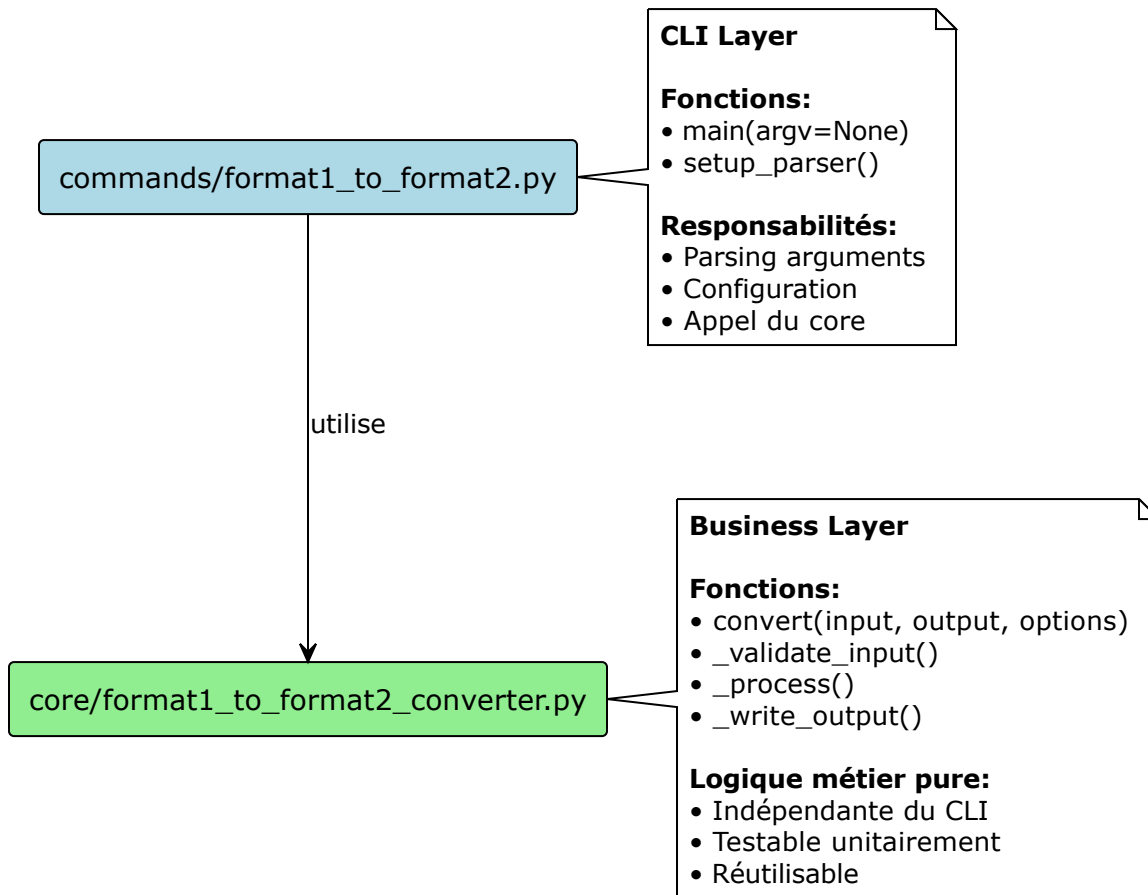
Flux de Résolution de Configuration



5. Configuration Hiérarchique

Hiérarchie de Configuration (CLI > YAML > ENV > Defaults)

Pattern des Modules de Conversion



Flux de Résolution de Configuration

Conventions de Nommage

html2md
Description = "HTML vers Markdown"

Format: <source>2<dest>;

- ✓ Toujours lowercase
- ✓ Pas de underscores
- ✓ Format source en premier
- ✓ "2" comme séparateur
- ✓ Format destination en dernier

Fichiers:

- commands/<source>2<dest>.py
- core/<source>2<dest>_converter.py

Desc

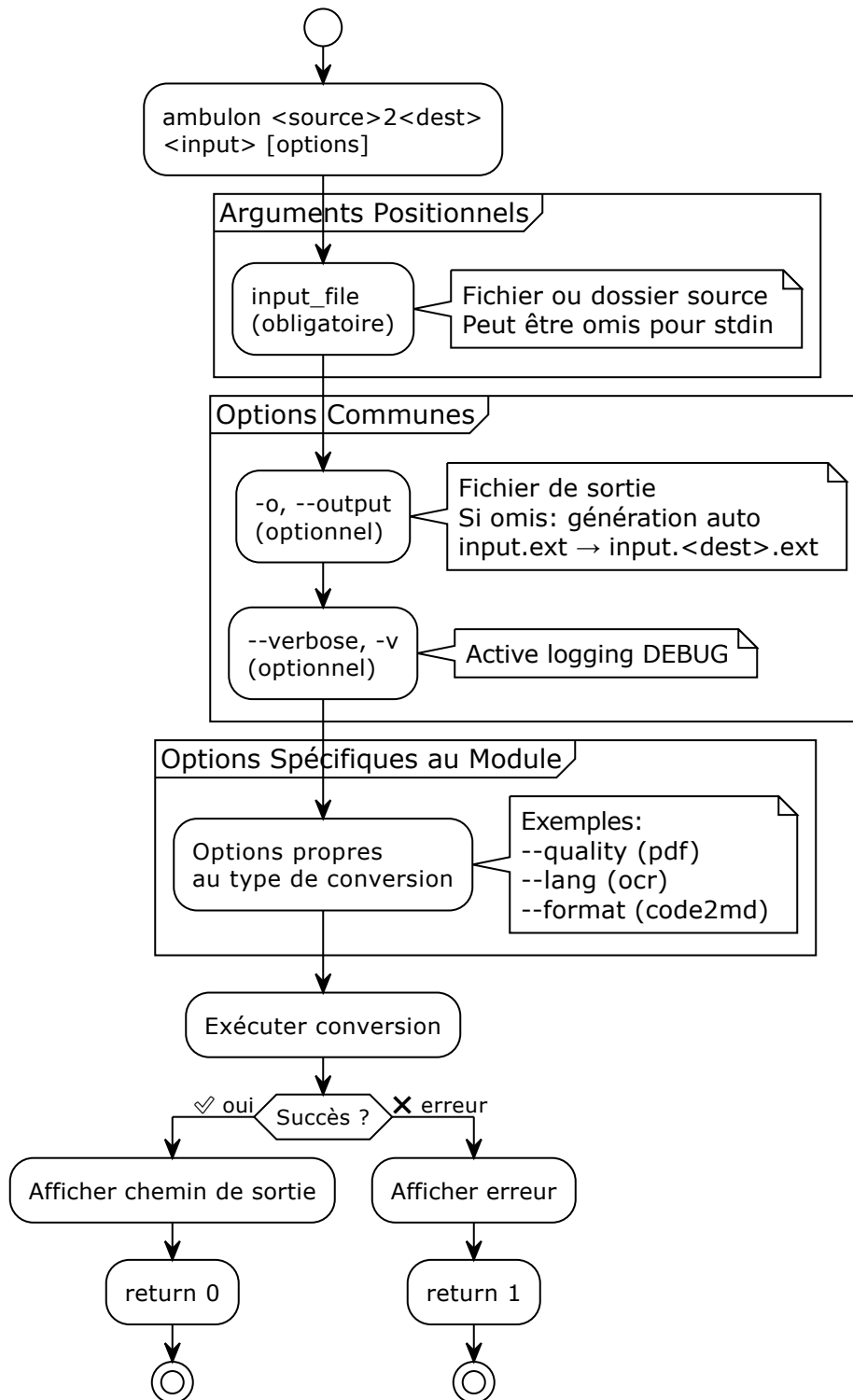
Desc



6. Pattern de Modules de Conversion

Architecture Standard des Modules

Interface CLI Standard pour Conversions



Naming Conventions

Gestion des Secrets - Workflow de Sécurité

✓ AUTORISÉ

Fichiers *.example

```
config/piag.yaml.example:  
  token: "VOTRE_TOKEN_ICI"
```

Variables ENV

```
export PIAG_TOKEN="real_token"  
export WIKISI_API_TOKEN="secret"
```

.gitignore

```
config/*.yaml  
!config/*.example  
.env  
credentials.json
```

Avant CHAQUE push:

```
git diff HEAD | grep -i "token"
```

Si secret détecté:

1. NE PAS PUSHER
2. Remplacer par placeholder
3. Amender le commit
4. Re-vérifier

✗ INTERDIT

Secrets dans Git (INTERDIT)

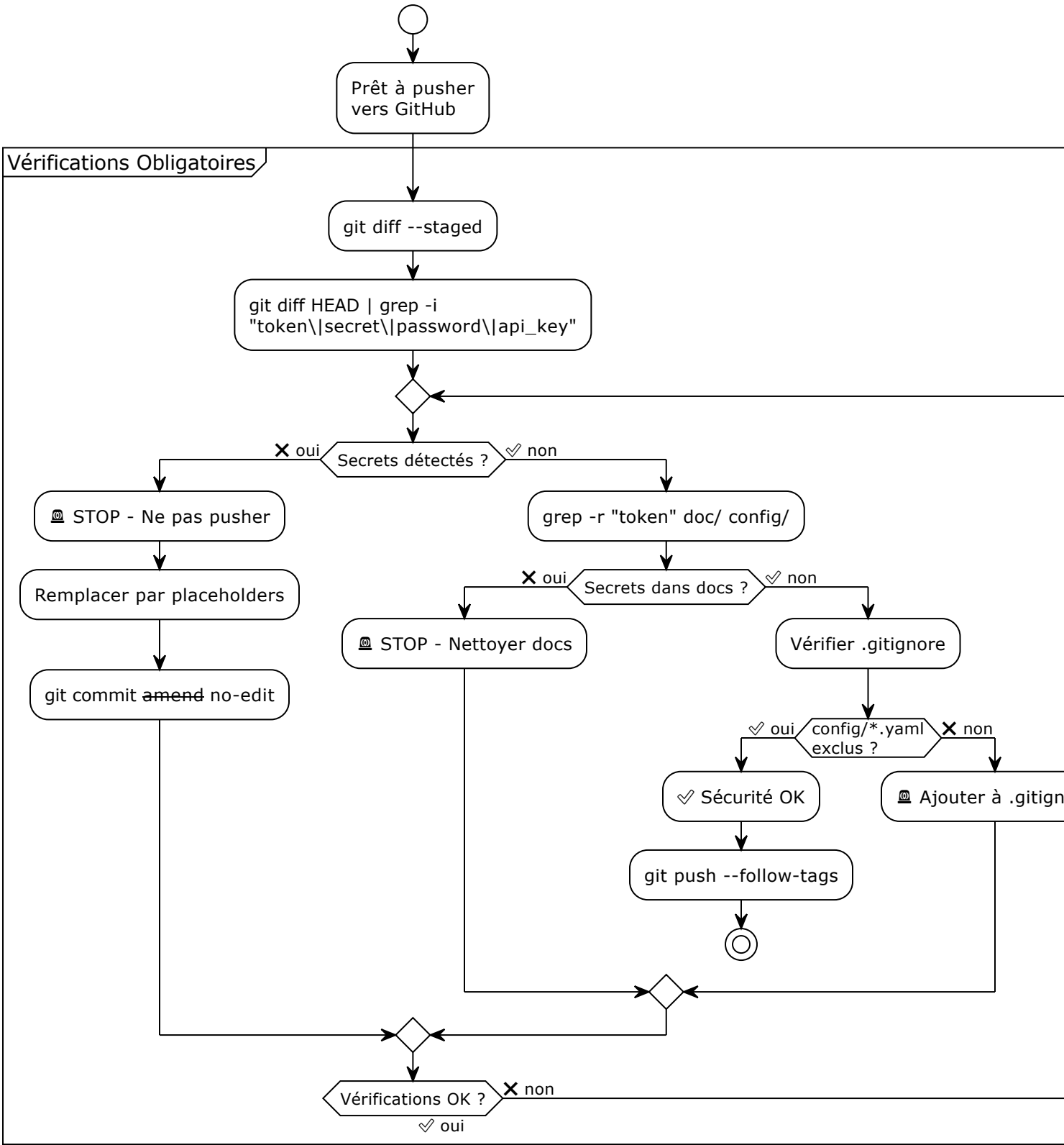
- ⊖ config/piag.yaml (v)
- ⊖ JWT tokens dans d
- ⊖ project_id réels dan
- ⊖ API keys dans exem

Si secret poussé:

1. Secret = COMPROMIS
2. Le révoquer IMMÉDIATEMENT
3. Générer nouveau s
4. NE PAS juste supprimer (reste accessible da

Interface CLI Unifiée

Checklist de Sécurité Avant Push



7. Gestion des Secrets

Règles de Sécurité Strictes

Structure du Projet Ambulon

ambulon/

```
.claude/
• GUIDELINES.md
• PROJECT.md
• settings.json

config/ (exemples versionnés)
• piag.yaml.example
• gitlab.yaml.example
• wikisi.yaml.example
• README.md

dist/ (généré, pas versionné)
• ambulon-3.0.2-py3-none-any.whl
• ambulon-3.0.2.tar.gz

dist-offline/ (80MB, versionné)
• ambulon-3.0.2-offline-install.zip
• 50 wheels de dépendances

doc/ (documentation PlantUML)
• git-workflow.md
• guidelines-illustrated.md

scripts/
• build_offline_package.py

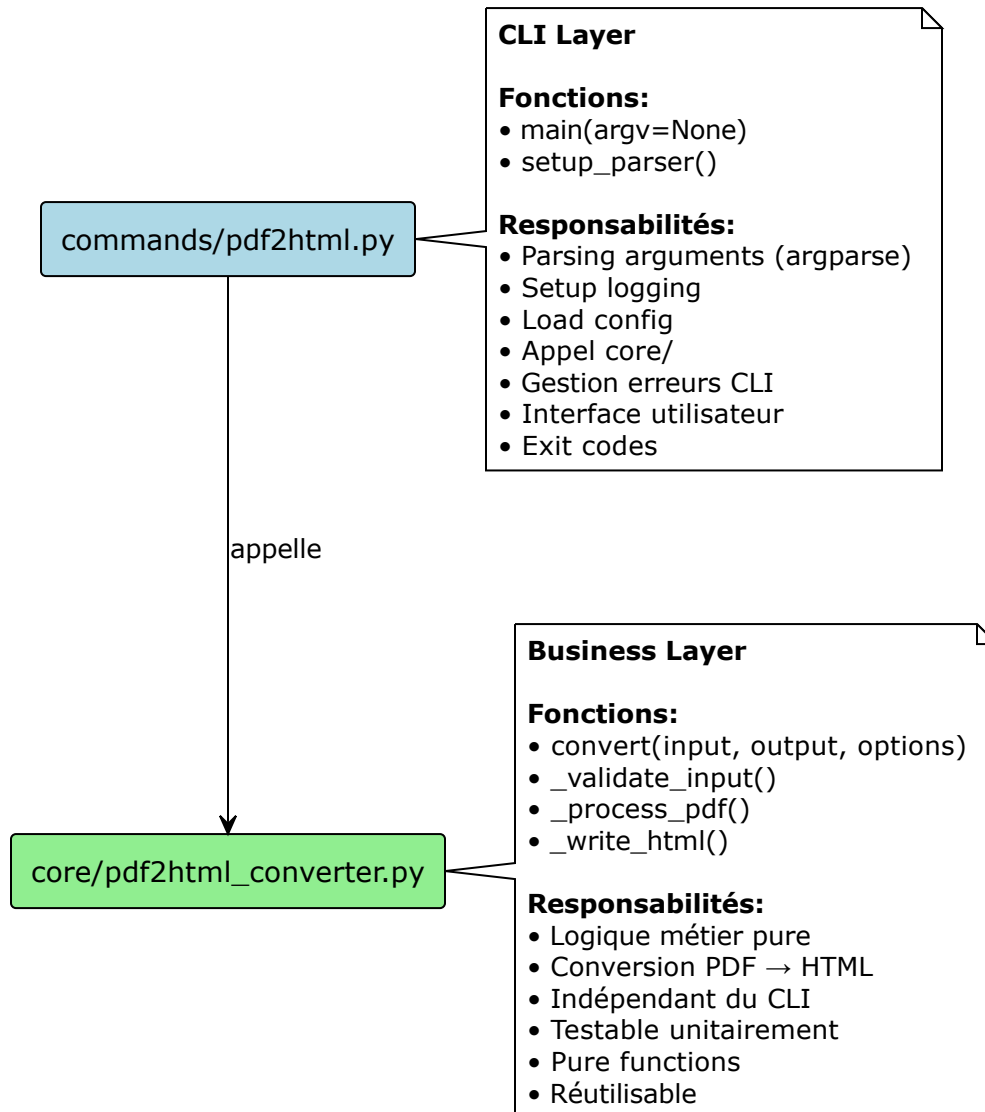
src/app/ (version = "3.0.2")
• cli/ (commands)
• conversion/
• piag/
• wikisi/
• init.py

Fichiers racine:
• pyproject.toml
• CHANGELOG.md
• .gitignore
• README.md

Principes:
✓ Séparation commands/core
✓ Config exemples versionnés
✓ Secrets dans .gitignore
✓ Documentation PlantUML
✓ Package offline sur preprod/prod
```

Checklist Pré-Push

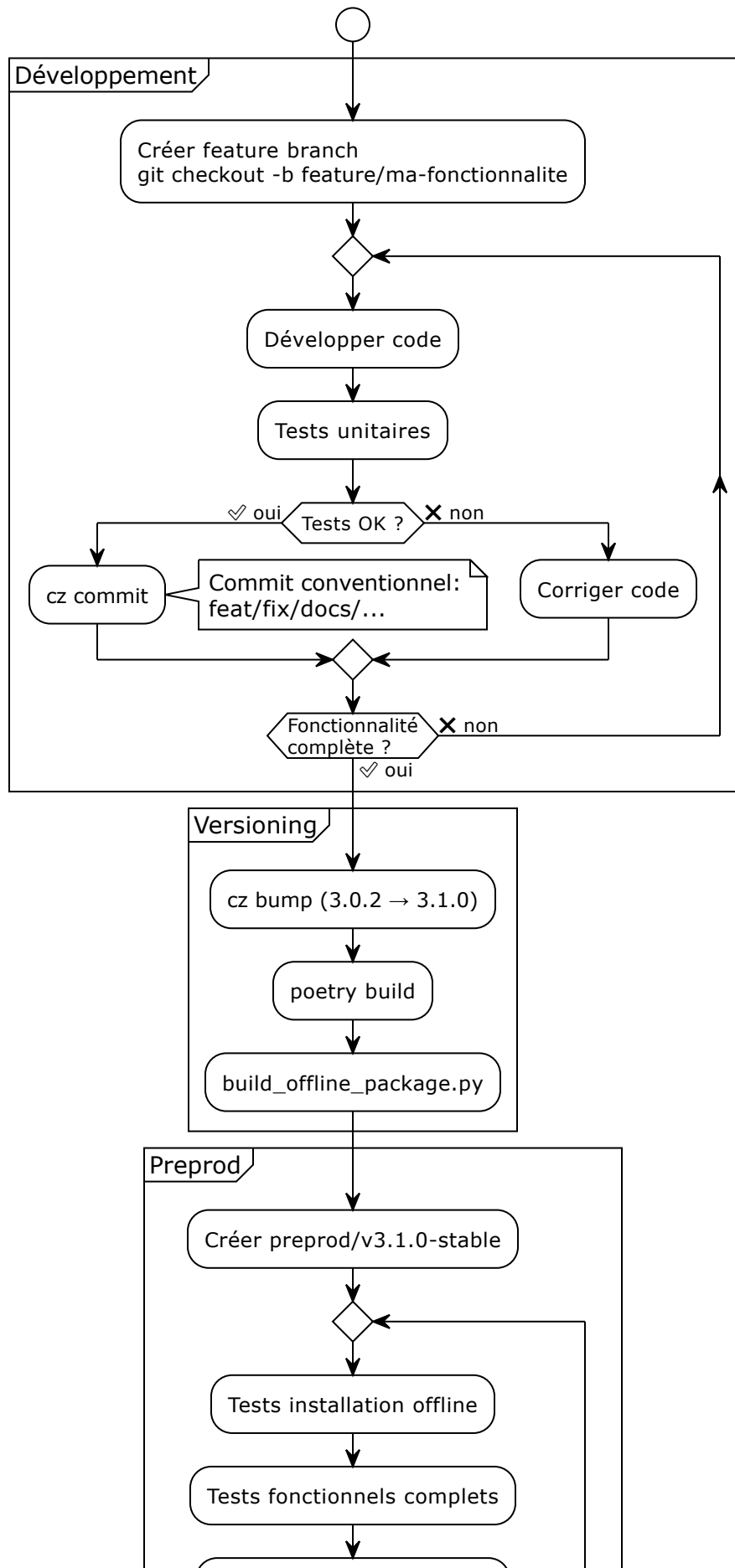
Séparation Commands / Core - Responsabilités



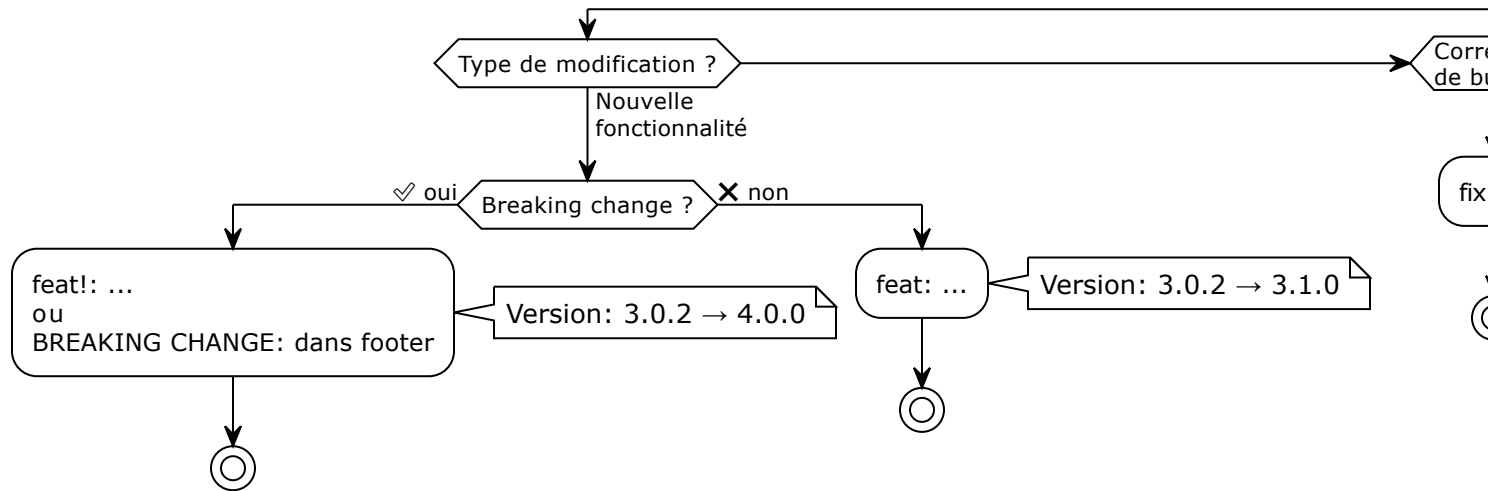
8. Structure du Projet

Arborescence Complète

Cycle de Vie Complet d'une Fonctionnalité



Séparation Commands / Core



9. Workflow Complet de Développement

Cycle de Vie d'une Fonctionnalité

```
PlantUML 1.2023.11

<b>This version of PlantUML is 872 days old, so you should
<b>consider upgrading from https://plantuml.com/download

[From string (line 6) ]

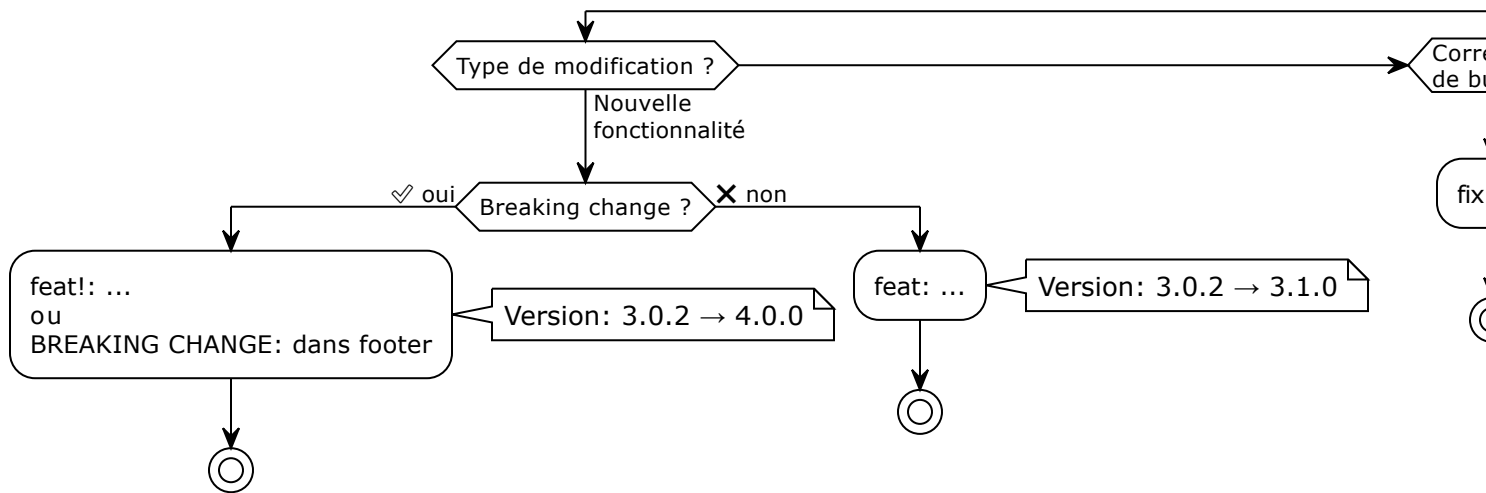
@startuml

...
... ( skipping 126 lines )
...
skinparam StereotypeE {
    BackgroundColor white
    BorderColor black
}
skinparam StereotypeI {
    BackgroundColor white
    BorderColor black
}
skinparam StereotypeN {
    BackgroundColor white
    BorderColor black
}
skinparam UseCaseStereoType {
    FontColor black
    FontName Verdana
}

title Principes Clés du Projet Ambulon

mindmap
Syntax Error?
```

Matrice de Décision - Quel Type de Commit ?



Génération des Diagrammes

Avec PlantUML

```
# Installation
sudo apt-get install plantuml # Linux
brew install plantuml         # macOS
choco install plantuml        # Windows

# Générer tous les diagrammes depuis ce fichier
plantuml doc/guidelines-illustrated.md

# Générer en SVG (meilleure qualité)
plantuml -tsvg doc/guidelines-illustrated.md

# Générer en PNG
plantuml -tpng doc/guidelines-illustrated.md
```

Avec le service en ligne

1. Copier un bloc PlantUML depuis ce fichier
 2. Aller sur <https://www.plantuml.com/plantuml/uml/>
 3. Coller le code
 4. Télécharger le diagramme généré
-

Résumé des Principes Clés

PlantUML 1.2023.11

**This version of PlantUML is 872 days old, so you should
consider upgrading from <https://plantuml.com/download>**

[From string (line 6)]

@startuml

...

... (skipping 126 lines)

...

```
skinparam StereotypeE {  
    BackgroundColor white  
    BorderColor black  
}
```

```
skinparam StereotypeI {  
    BackgroundColor white  
    BorderColor black  
}
```

```
skinparam StereotypeN {  
    BackgroundColor white  
    BorderColor black  
}
```

```
skinparam UseCaseStereoType {  
    FontColor black  
    FontName Verdana  
}
```

title Principes Clés du Projet Ambulon

mindmap

Syntax Error?

Auteur: Hervé Marchal herve.marchal@hotmail.fr

Version: 3.0.2

Date: 2026-01-31

Licence: MIT

Index des Diagrammes

1. **Architecture CLI**

- Typage vs argparse
- Pattern CLI obligatoire
- Flux d'exécution CLI

2. **Workflow Git**

- Architecture des branches
- Cycle de vie des branches
- Processus de release
- Gestion des bugs
- Timeline exemple

3. **Versioning**

- SemVer workflow
- Types de commits
- Matrice de décision commits

4. **Configuration**

- Hiérarchie de configuration
- Flux de résolution

5. **Modules de Conversion**

- Architecture standard
- Naming conventions
- Interface CLI unifiée

6. **Sécurité**

- Règles secrets
- Checklist pré-push

7. **Structure Projet**

- Arborescence complète
- Séparation commands/core

8. **Workflow Développement**

- Cycle de vie feature
- Matrice de décision

9. **Résumé**

- Mindmap principes clés